Diagnose and fix a failing deployment

In this exercise, you troubleshoot a failing container app and apply targeted fixes. You use revision status, logs, and the Azure CLI to isolate deployment issues. This workflow is common in AI solutions because startup behavior changes frequently when you update models and dependencies.

Tasks performed in this exercise:

- Deploy a mock AI document processing API as a container app
- Introduce and diagnose a missing environment variable error
- Introduce and diagnose an ingress configuration issue
- Query Log Analytics for historical troubleshooting data

This exercise takes approximately **30** minutes to complete.

> **Important:** Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can sign up for one.
- Visual Studio Code on one of the supported platforms.
- The latest version of the Azure CLI.
- Optional: Python 3.12 or greater.

Download project starter files and deploy Azure services

In this section you download the project starter files and use a script to deploy the necessary services to your Azure subscription. The Azure Container Registry and Container Apps environment deployment takes a few minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

   Code Copy

   ```
   https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aca-manage-python.zip
   ```

2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.

3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.

4. Open the *azdeploy.py* deployment script and change the two values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

   Code Copy

   ```
   "<your-resource-group-name>" # Resource Group name
   "<your-azure-region>" # Azure region for the resources
   ```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.

6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

   Code Copy

   ```
   az login
   ```

7. Run the following command to ensure you have the **containerapp** extension for Azure CLI.

Code Copy

```
az extension add --name containerapp
az extension add --name log-analytics
```

8. Run the following commands to ensure your subscription has the necessary resource providers for the exercise.

Code Copy

```
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace Microsoft.ContainerRegistry
```

Create resources in Azure

In this section you run the deployment script to deploy the necessary services to your Azure subscription.

1. Make sure you are in the root directory of the project and run the following command in the terminal to launch the deployment script. The deployment script deploys ACR and creates the environment variable files needed for the exercise.

Code Copy

```
python azdeploy.py
```

2. When the script is running, enter **1** to launch the **Create Azure Container Registry and build container image** option. This option creates the ACR service and uses ACR Tasks to build and push the image to the registry.

3. When the previous operation is finished, enter **2** to launch the **Create Container Apps environment** options. Creating the environment is necessary before deploying the container.

4. When the previous operation is finished, enter **3** to launch the **Deploy the container app and configure secrets** option.

> **Note:** A file containing environment variables is created after the container app is created. You use these variables throughout the exercise.

5. When the previous operation is finished, enter **5** to exit the deployment script.

6. Run the appropriate command to load the environment variables into your terminal session from the file created in a previous step.

Bash

Code Copy

```
source .env
```

PowerShell

Code Copy

```
. .\.env.ps1
```

> **Note:** Keep the terminal open. If you close it and create a new terminal, you might need to run the command to create the environment variable again.

7. Run the following command to retrieve the app FQDN and store the result to a variable.

Bash

Code Copy

```
FQDN=$(az containerapp show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --query properties.configuration.ingress.fqdn -o tsv)

echo "$FQDN"
```

PowerShell

```
$FQDN = az containerapp show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --query properties.configuration.ingress.fqdn -o tsv

Write-Output $FQDN
```

8. Run the following command to call the default endpoint to verify the app is running. The command should return some JSON. Look for the **model.name** field, it should be set to **gpt-4o-mini**.

Bash

```
curl -s "https://$FQDN/"
```

PowerShell

```
Invoke-RestMethod -Uri "https://$FQDN/"
```

Diagnose a missing environment variable

When a container app depends on an environment variable that isn't set, the app may fail to start or behave unexpectedly. In this section, you remove a required environment variable and observe the symptoms.

1. Run the following command to update the container app to remove the `MODEL_NAME` environment variable.

Bash

```
az containerapp update -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --remove-env-vars MODEL_NAME
```

PowerShell

```
az containerapp update -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --remove-env-vars MODEL_NAME
```

2. Run the following command to list revisions to confirm a new revision was created. Look for a new revision with a higher suffix number (for example, **ai-api--0000002**) and **TrafficWeight** of **100**, indicating it's now receiving all traffic.

Bash

```
az containerapp revision list -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP -o table
```

PowerShell

```
az containerapp revision list -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o
table
```

3. Run the following command to check the root endpoint to observe the symptom from the API consumer's perspective. The **model.name** field now shows the default value of **not-configured** instead of the configured value.

Bash

```
curl -s "https://$FQDN/" | jq .model
```

PowerShell

Code Copy

```
(Invoke-RestMethod -Uri "https://$FQDN/").model
```

4. Run the following command to diagnose the root cause by viewing the container app's configuration. Run the following command to confirm the **MODEL_NAME** environment variable is missing.

Bash

Code Copy

```
az containerapp show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --query "properties.template.containers[0].env" -o table
```

PowerShell

Code Copy

```
az containerapp show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --query "properties.template.containers[0].env" -o table
```

5. Run the following command to fix the issue by adding the `MODEL_NAME` environment variable back.

Bash

Code Copy

```
az containerapp update -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --set-env-vars MODEL_NAME=$MODEL_NAME
```

PowerShell

Code Copy

```
az containerapp update -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --set-env-vars MODEL_NAME=$env:MODEL_NAME
```

6. Run the following command to verify the fix by checking the root endpoint again. This confirms the application now behaves correctly from an API consumer's perspective. The response should now show the configured model name.

Bash

Code Copy

```
curl -s "https://$FQDN/" | jq .model
```

PowerShell

Code Copy

```
(Invoke-RestMethod -Uri "https://$FQDN/").model
```

You diagnosed and fixed a missing environment variable. Next, you diagnose a secret an ingress issue.

Diagnose an ingress configuration issue

Container Apps uses the **target-port** setting to route traffic to your container. If the port doesn't match what your application listens on, requests fail. In this section, you introduce a port mismatch.

1. Run the following command to update the container app to use the wrong target port.

Bash

Code Copy

```
az containerapp ingress update -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --target-port 3000
```

PowerShell

Code Copy

```
az containerapp ingress update -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --target-port 3000
```

2. Run the following command to try to access the health endpoint to observe the symptom from an API consumer's perspective.

Bash

Code Copy

```
curl -s "https://$FQDN/health"
```

PowerShell

Code Copy

```
Invoke-RestMethod -Uri "https://$FQDN/health"
```

The request fails or times out because Container Apps is routing traffic to port 3000, but the application listens on port 8000.

3. Run the following command to diagnose the root cause by checking the current ingress configuration. Notice the **targetPort** is set to 3000.

Bash

Code Copy

```
az containerapp show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --query "properties.configuration.ingress" -o yaml
```

PowerShell

Code Copy

```
az containerapp show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --query "properties.configuration.ingress" -o yaml
```

4. Run the following command to check the container logs to see if the application is running. You should see gunicorn startup messages indicating the app is listening on port 8000, confirming the mismatch.

Bash

Code Copy

```
az containerapp logs show -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP
```

PowerShell

Code Copy

```
az containerapp logs show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP
```

5. Run the following command to fix the ingress configuration by setting the correct target port.

Bash

Code Copy

```
az containerapp ingress update -n $CONTAINER_APP_NAME -g $RESOURCE_GROUP \
    --target-port 8000
```

PowerShell

```
az containerapp ingress update -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
    --target-port 8000
```

6. Run the following command to verify the fix by calling the health endpoint. This confirms the application is accessible from an API consumer's perspective. You should see **{"status":"healthy"}**.

   Bash

```
curl -s "https://$FQDN/health"
```

   PowerShell

```
Invoke-RestMethod -Uri "https://$FQDN/health"
```

You diagnosed and fixed an ingress configuration issue. Next, you learn how to query historical logs.

Query Log Analytics for historical troubleshooting

Console logs shown by **az containerapp logs show** are recent only. For historical troubleshooting, logs persist in the Log Analytics workspace associated with your Container Apps environment.

1. Run the following command to get the Log Analytics workspace ID from the Container Apps environment.

   Bash

```
WORKSPACE_ID=$(az containerapp env show -n $ACA_ENVIRONMENT -g $RESOURCE_GROUP \
    --query properties.appLogsConfiguration.logAnalyticsConfiguration.customerId -o
tsv)

echo "Workspace ID: $WORKSPACE_ID"
```

   PowerShell

```
$WORKSPACE_ID = az containerapp env show -n $env:ACA_ENVIRONMENT -g $env:RESOURCE_GROUP
`
    --query properties.appLogsConfiguration.logAnalyticsConfiguration.customerId -o tsv

Write-Output "Workspace ID: $WORKSPACE_ID"
```

2. Run the following command to query the console logs for your container app. This returns the last 20 log entries showing timestamp and message.

   Bash

```
az monitor log-analytics query -w $WORKSPACE_ID \
    --analytics-query "ContainerAppConsoleLogs_CL | where ContainerAppName_s ==
'$CONTAINER_APP_NAME' | project TimeGenerated, Log_s | order by TimeGenerated desc |
take 20" \
    -o table
```

   PowerShell

```
az monitor log-analytics query -w $WORKSPACE_ID `
    --analytics-query "ContainerAppConsoleLogs_CL | where ContainerAppName_s ==
'$env:CONTAINER_APP_NAME' | project TimeGenerated, Log_s | order by TimeGenerated desc
| take 20" `
    -o table
```

[!NOTE] Log Analytics data may take a few minutes to appear after events occur. If you don't see recent logs, wait a few minutes and try again.

3. Run the following command to query for error-level logs specifically.

Bash

Code Copy

```
az monitor log-analytics query -w $WORKSPACE_ID \
    --analytics-query "ContainerAppConsoleLogs_CL | where ContainerAppName_s ==
'$CONTAINER_APP_NAME' and Log_s contains 'error' | order by TimeGenerated desc | take
20" \
    -o table
```

PowerShell

Code Copy

```
az monitor log-analytics query -w $WORKSPACE_ID `
    --analytics-query "ContainerAppConsoleLogs_CL | where ContainerAppName_s ==
'$env:CONTAINER_APP_NAME' and Log_s contains 'error' | order by TimeGenerated desc |
take 20" `
    -o table
```

These queries help you investigate issues that occurred in the past, even after container restarts or revision changes.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

Code Copy

```
az group delete --name <rg-name> --no-wait --yes
```

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues during this exercise, try these steps:

Container app not responding

- Check if the revision is active using **az containerapp revision list**
- Verify ingress is configured using **az containerapp show**

Cannot see logs

- Console logs are recent only. Use Log Analytics for historical data.
- Log Analytics data may take 2-5 minutes to appear.

Environment variables not taking effect

- Container Apps creates a new revision when you change environment variables. Verify the new revision is active.

- Use **--replace-env-vars** carefully—it replaces all environment variables, not just the ones you specify.